Міністерство освіти і наук України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №2
Варіант 1
На тему
“Функції вищого порядку, lambda, композиція”

Виконав:
Студент групи ФЕП-21
Бардак Д. Т

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи: побудувати конвеєр перетворень над списком записів (records) без мутацій.

Теоретичні відомості:

•

map / filter / reduce: семантика та ідіоми

2.1. map

map() - це вбудована функція Python, яка застосовує задану функцію до кожного елемента

послідовності.

Загальний вигляд: map(function, iterable)

де function - функція, яку треба застосувати, а iterable - список, кортеж або інший ітерований

об'єкт.

Наприклад:

```
xs = [" A ", " b ", "C "]
result = map(str.upper, xs)
print(list(result))
```

Результат:

```
[' A ', ' B ', 'C ']
```

Важлива особливість: map() ледачий. Він не створює одразу новий список з усіма

результатами, а повертає ітератор, який обчислює значення тоді, коли вони потрібні.

 Ледачий (повертає ітератор), тому не створює копій.

 В Python генератори часто читабельніші: [f(x) for x in xs].

```
xs = [" A ", " b ", "C "]
normalized = list(map(str.strip, map(str.upper, xs)))
# або: [x.strip().upper() for x in xs]
```

2.2. filter

 Приймає предикат: функцію, що повертає True/False.

 Часто зручно комбінувати з itertools (dropwhile, takewhile).

```
nums = [0, -2, 5, 10, -1]
positive = list(filter(lambda x: x > 0, nums)) # [5, 10]
```

filter() - це вбудована функція Python, яка відбирає з послідовності лише ті елементи,

які задовольняють задану умову.
Загальний вигляд:
filter(function, iterable)
де function - функція-предикат, яка для кожного елемента повертає True або False, а
iterable - список, кортеж або інший ітерований об'єкт.
nums = [0, -2, 5, 10, -1]
positive = list(filter(lambda x: x > 0, nums))
print(positive)
Результат:
[5, 10]
Тут:
lambda x: x > 0 - це предикат, тобто функція, яка перевіряє умову.
Для кожного числа відбувається така перевірка:
x x > 0 Результат
0 False відкидається
-2 False відкидається
5 True залишається
10 True залишається
-1 False відкидається
Тому: positive = [5, 10]

Хід роботи

У ході виконання лабораторної роботи розроблено програму для обробки даних клієнтів інтернет-магазину з використанням функціонального підходу. Архітектуру рішення побудовано із суворим розділенням чистого обчислювального ядра в модулі core.py та операцій вводу-виводу в app.py. Визначивши типізовану структуру запису Record на базі TypedDict, було реалізовано набір чистих трансформацій без мутації вхідних даних із застосуванням лямбда-виразів, генераторів та функцій вищого порядку. За допомогою композиції (compose) ці функції інтегровано в єдиний конвеєр перетворень, що поетапно забезпечує фільтрацію повнолітніх осіб, підрахунок суми покупок, застосування бонусного коефіцієнта для заданого міста, сортування за спаданням та вибірку Top-N клієнтів. Додатково реалізовано функцію агрегації даних на основі reduce для розрахунку кількості записів, загальної та середньої сум. Надійність створеного конвеєра, правильність агрегації та відсутність побічних ефектів у вигляді мутацій оригінальних даних підтверджено автоматизованими

тестами у фреймворку pytest, а якість і сувору типізацію коду успішно верифіковано інструментами black, ruff та mypy



```python
from collections.abc import Callable, Iterable, Iterator
from functools import reduce
from itertools import islice
from typing import Any, TypedDict


class Record(TypedDict, total=False):
    id: int
    name: str
    age: int
    city: str
    purchases: list[float]
    total: float


def compose(*funcs: Callable[[Any], Any]) -> Callable[[Any], Any]:
    def _c(x: Any) -> Any:
        for f in reversed(funcs):
            x = f(x)
        return x

    return _c


def only_adults(min_age: int = 18) -> Callable[[Iterable[Record]], Iterator[Record]]:
    return lambda recs: (r for r in recs if int(r.get("age", 0)) >= min_age)


def with_total() -> Callable[[Iterable[Record]], Iterator[Record]]:
    return lambda recs: (
        {**r, "total": float(sum(r.get("purchases", [])))} for r in recs
    )


def boost_city(
    city: str, factor: float
) -> Callable[[Iterable[Record]], Iterator[Record]]:
    def _f(recs: Iterable[Record]) -> Iterator[Record]:
        for r in recs:
            t = float(r.get("total", 0.0))
            new_total = t * factor if r.get("city") == city else t
            yield {**r, "total": new_total}

    return _f


def normalize_names() -> Callable[[Iterable[Record]], Iterator[Record]]:
    return lambda recs: (
        {**r, "name": str(r.get("name", "")).strip().title()} for r in recs
    )


def sort_by_total_desc() -> Callable[[Iterable[Record]], list[Record]]:
    return lambda recs: sorted(
        recs, key=lambda r: (-float(r.get("total", 0.0)), int(r.get("id", 0)))
```



```
    )


def take(n: int) -> Callable[[Iterable[Record]], list[Record]]:
    return lambda recs: list(islice(recs, n))


def build_pipeline(
    top_n: int = 2, city: str = "Delhi", factor: float = 1.1
) -> Callable[[Iterable[Record]], list[Record]]:
    return compose(
        take(top_n),
        sort_by_total_desc(),
        boost_city(city, factor),
        with_total(),
        only_adults(18),
        normalize_names(),
    )


def reduce_stats(recs: Iterable[Record]) -> dict[str, float]:
    init_acc = {"count": 0.0, "sum_total": 0.0}

    acc = reduce(
        lambda a, r: {
            "count": a["count"] + 1,
            "sum_total": a["sum_total"] + float(r.get("total", 0.0)),
        },
        recs,
        init_acc,
    )

    count = acc["count"]
    avg = acc["sum_total"] / count if count > 0 else 0.0

    return {
        "count": count,
        "sum_total": acc["sum_total"],
        "avg_total": avg,
    }
```

Рис. 1-4 Файл core.py

```python
import argparse

from core import Record, build_pipeline, reduce_stats


def load_data(filepath: str) -> list[Record]:
    return [
        {
            "id": 1,
            "name": "john doe",
            "age": 25,
            "city": "Delhi",
            "purchases": [100.0, 50.0],
        },
        {
            "id": 2,
            "name": "jane smith",
            "age": 17,
            "city": "London",
            "purchases": [300.0],
        },
        {"id": 3, "name": "bob", "age": 30, "city": "Paris", "purchases": [200.0]},
        {"id": 4, "name": "alice", "age": 22, "city": "Delhi", "purchases": [200.0]},
    ]


def main() -> None:
    parser = argparse.ArgumentParser(description="Обробка записів клієнтів.")
    parser.add_argument("--top", type=int, default=2, help="Кількість записів у Top-N")
    parser.add_argument(
        "--boost-city", type=str, default="Delhi", help="Місто для бонусу"
    )
    parser.add_argument("--factor", type=float, default=1.1, help="Коефіцієнт бонусу")
    args = parser.parse_args()

    raw_data = load_data("sample_data.json")

    pipeline = build_pipeline(top_n=args.top, city=args.boost_city, factor=args.factor)

    processed_data = pipeline(raw_data)

    stats = reduce_stats(processed_data)

    print(f"=== TOP {args.top} КЛІЄНТІВ ===")
    for rec in processed_data:
        print(
            f"ID: {rec['id']} | Name: {rec['name']} | "
            f"City: {rec['city']} | Total: {rec['total']:.2f}"
        )

    print("\n=== АГРЕГОВАНА СТАТИСТИКА ===")
    print(f"Кількість (count):     {stats['count']}")
    print(f"Сума (sum_total):      {stats['sum_total']:.2f}")
    print(f"Середнє (avg_total):   {stats['avg_total']:.2f}")


if __name__ == "__main__":
    main()
```

Рис. 5-7 Файл app.py

```python
from copy import deepcopy

import pytest

from core import Record, build_pipeline, reduce_stats


@pytest.fixture
def sample_records() -> list[Record]:
    return [
        {"id": 1, "name": "john doe", "age": 20, "city": "Delhi", "purchases": [100.0]},
        {"id": 2, "name": "jane", "age": 16, "city": "Delhi", "purchases": [500.0]},
        {"id": 3, "name": "bob", "age": 25, "city": "London", "purchases": [200.0]},
    ]


def test_pipeline_top_n(sample_records: list[Record]) -> None:
    pipeline = build_pipeline(top_n=2, city="Delhi", factor=1.5)
    result = pipeline(sample_records)

    assert len(result) == 2

    assert result[0]["id"] == 3
    assert result[0]["total"] == 200.0

    assert result[1]["id"] == 1
    assert result[1]["total"] == 150.0

    assert result[1]["name"] == "John Doe"


def test_no_mutation(sample_records: list[Record]) -> None:
    original = deepcopy(sample_records)
    pipeline = build_pipeline(top_n=5, city="Delhi", factor=2.0)

    pipeline(sample_records)

    assert sample_records == original


def test_reduce_stats() -> None:
    recs: list[Record] = [
        {"id": 1, "total": 100.0},
        {"id": 2, "total": 300.0},
    ]

    stats = reduce_stats(recs)

    assert stats["count"] == 2.0
    assert stats["sum_total"] == 400.0
    assert stats["avg_total"] == 200.0
```

Рис. 7-8 Файл test_core.py

```
/home/ritokk/Documents/fp/lab02/.venv/bin/python /home/ritokk/Documents/fp/lab02/lab02/app.py
=== TOP 2 КЛІЄНТІВ ===
ID: 4 | Name: Alice | City: Delhi | Total: 220.00
ID: 3 | Name: Bob | City: Paris | Total: 200.00

=== АГРЕГОВАНА СТАТИСТИКА ===
Кількість (count):     2.0
Сума (sum_total):      420.00
Середнє (avg_total):   210.00

Process finished with exit code 0
```

Рис. 9 Запуск програми

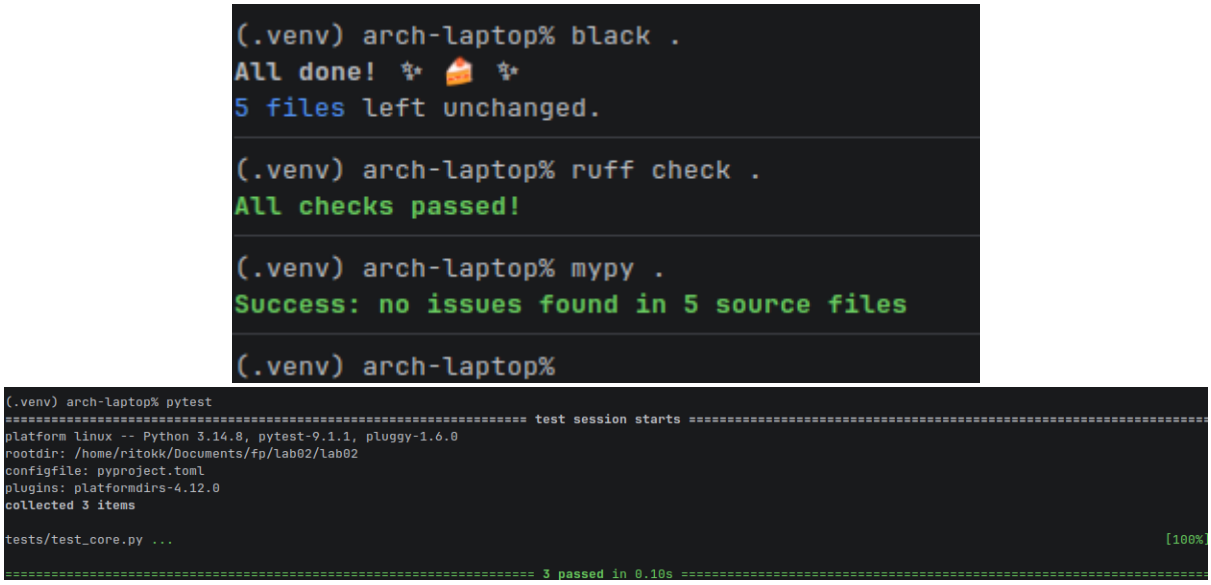


Рис.10-11 Успішно виконані тести

Висновок:

Під час виконання лабораторної роботи було успішно побудовано функціональний конвеєр перетворень над списком записів без мутації вхідних даних. Здобуто практичні навички роботи з чистими функціями, лямбда-виразами та функціями вищого порядку (map, filter, reduce). Завдяки застосуванню анотацій типів, Callable-об'єктів та функції compose, вдалося досягти високої гнучкості й референтної прозорості коду, ізолювавши всі побічні ефекти (I/O) в окремому модулі. Автоматизовані тести та перевірки якості коду підтвердили надійність і правильність реалізованого рішення.